

EXAMLYTIC

Developer Blueprint Version 2.0

Complete Page-by-Page Development Guide with SuperAdmin Role

SuperAdmin	Admin	Teacher	Student
-------------------	--------------	----------------	----------------

1. Roles Overview

Examlytic has 4 roles. Each role has a completely separate dashboard and set of permissions. No role can access another role's pages.

Role	Who	Dashboard Route	How Account is Created
SuperAdmin	You (the developer/owner)	/superadmin/dashboard	Seed script one time only
Admin	School / College management	/admin/dashboard	SuperAdmin creates from the dashboard
Teacher	Subject teachers	/teacher/dashboard	Admin invites via email
Student	Students	/student/dashboard	Teacher invites OR class code join



Important for Developers

Role is stored in the JWT token after login. On every protected route, middleware checks the token role and redirects unauthorized users. Never trust client-side role checks alone — always verify on the backend.

2. App Entry Point & Authentication Flow

2.1 What Happens When App Runs

When developer runs the app (npm run dev), this is the exact flow:

1. Browser opens → localhost:5173 → Redirects to /login
2. User enters email + password → POST /api/auth/login
3. Backend checks credentials → returns JWT token with { id, role }
4. Frontend stores token in httpOnly cookie or localStorage
5. Frontend reads role from token → redirects to correct dashboard:

Role in Token	Redirects To
superadmin	/superadmin/dashboard
admin	/admin/dashboard
teacher	/teacher/dashboard
student	/student/dashboard

2.2 Login Page — /login

Element	Details
Logo	Examlytic logo top center
Email field	Text input — required
Password field	Password input with show/hide toggle — required
Login button	POST /api/auth/login — on success redirect by role
Error message	Show 'Invalid email or password' — do NOT say which one is wrong
No signup link	There is NO public signup link on this page — by design
Forgot password	Skip for now / If possible please add

No Public Signup

There is absolutely no signup page accessible from the login page. All accounts are created via invitation or seed script. This is a security feature. Mention this to your supervisor.

2.3 Invitation Signup Page /signup

This page is only accessible via an invitation link sent by email. Direct URL access without a valid token shows an 'Invalid or expired link' error.

Element	Details
URL format	/signup?token=abc123&email=user@gmail.com
Email field	Pre-filled from URL param — read only, cannot change
Name field	User types their full name
Password field	User sets their password
Confirm Password	Must match password
Submit button	POST /api/auth/signup-invite — validates token, creates account
Token expiry	Token expires in 48 hours — show error if expired

2.4 Seed Script First Time Setup

```
// backend/scripts/seed.js — Run ONCE: node scripts/seed.js
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');
const User = require('../models/User');

mongoose.connect(process.env.MONGO_URI);

async function seed() {
  const exists = await User.findOne({ role: 'superadmin' });
  if (exists) { console.log('SuperAdmin exists!'); process.exit(); }
}
```

```
await User.create({
  name: 'Super Admin',
  email: 'superadmin@examlytic.com',
  password: await bcrypt.hash('Super@123', 10),
  role: 'superadmin'
});
console.log('SuperAdmin created!');
process.exit();
}
seed();
```

Run this once

After running seed.js, login with superadmin@examlytic.com and Super@123. Immediately change the password from the SuperAdmin settings page. Delete or disable seed.js before deploying to production.

3. SuperAdmin Panel

Role: *superadmin* | Route prefix: */superadmin* | Access: *You only*

3.1 What SuperAdmin Sees

SuperAdmin is the god-level user. He manages the entire platform all admins, all institutions. He does NOT manage teachers or students directly. That is the Admin's job.

3.2 SuperAdmin Sidebar Navigation

Nav Item	Route	What It Shows
Dashboard	/superadmin/dashboard	Stats overview total admins, total institutions, total users
Admins	/superadmin/admins	List of all admin accounts created
Create Admin	/superadmin/admins/create	Form to create a new admin account
Settings	/superadmin/settings	SuperAdmin profile settings, change password
Logout	—	Clear token, redirect to /login

3.3 SuperAdmin Dashboard Page /superadmin/dashboard

This is the home page after SuperAdmin logs in. It shows platform-wide statistics.

UI Element	Details
Stats Card 1	Total Admins — number of admin accounts created
Stats Card 2	Total Teachers — count across all institutions
Stats Card 3	Total Students — count across all institutions
Stats Card 4	Total Exams — total exams created on platform
Recent Admins Table	Last 5 admins created — Name, Institution, Email, Created Date, Status badge
Quick Action Button	'+ Create New Admin' button — links to /superadmin/admins/create

3.4 Admins List Page /superadmin/admins

SuperAdmin can see and manage all admin accounts here.

Column	Details
Name	Admin's full name

Column	Details
Institution Name	School/College name they represent
Email	Admin's email address
Phone	Admin's phone number
Created Date	When this admin was created
Status	Active (green badge) / Suspended (red badge)
Actions	View Details Suspend Delete

Suspend vs Delete

Suspend = Admin cannot login but data is preserved. Delete = permanent removal of admin and all their data (classes, teachers, students, exams). Always show a confirmation popup before delete.

3.5 Create Admin Page /superadmin/admins/create

SuperAdmin fills this form to give a new school/college access to Examlytic.

Field	Type	Validation
Admin Full Name	Text input	Required, min 3 chars
Institution Name	Text input	Required — e.g., Punjab College Lahore
Email Address	Email input	Required, valid email, must be unique
Phone Number	Text input	Required, valid Pakistani format
Qualification	Text input	Optional — e.g., MBA, PhD
Password	Password input	Required, min 8 chars, 1 uppercase, 1 number
Confirm Password	Password input	Must match password
Submit Button	Button	POST /api/superadmin/admins — creates account directly (no email invite needed for admin)

No Email Invite for Admin

SuperAdmin creates admin directly — no invitation email. SuperAdmin tells the admin their credentials verbally or via a separate private message. This keeps it simple for FYP.

3.6 SuperAdmin Settings /superadmin/settings

Field	Details
Change Name	Update SuperAdmin display name
Change Email	Update SuperAdmin email — requires current password
Change Password	Old password + New password + Confirm — POST /api/superadmin/change-password

4. Admin Panel

Role: admin | Route prefix: /admin | Access: School/College management

4.1 How Admin is Different Now (SuperAdmin Change)

Previously there was no SuperAdmin. Now Admin is created BY SuperAdmin. The only change to the Admin panel is:

- Admin CANNOT create other admins — only SuperAdmin can do that
- Admin cannot see other institutions' data — only their own
- Everything else in Admin panel remains the same as Version 1

4.2 Admin Sidebar Navigation

Nav Item	Route	Purpose
Dashboard	/admin/dashboard	Overview of institution stats
Teachers	/admin/teachers	Manage teachers — view + invite
Students	/admin/students	View all enrolled students
Settings	/admin/settings	Institution details, change password
Logout	—	Clear token, go to /login

4.3 Admin Dashboard /admin/dashboard

UI Element	Details
Stats Card 1	Total Teachers in this institution
Stats Card 2	Total Students enrolled across all classes
Stats Card 3	Total Active Exams right now
Stats Card 4	Total Classes created
Teachers Table (preview)	Last 5 teachers added — Name, Email, Phone, Status
Quick Action	'+ Add Teacher' button → opens invite flow

4.4 Teachers Page /admin/teachers

Same as shown in Image 1 of your screenshots. Table with:

- Name, Qualification, Email, Phone columns
- 'Add Staff' button top right → opens modal with email input → sends invitation email
- Actions column: Remove Teacher (with confirmation popup)

- Status badge: Active / Pending Invite (if teacher hasn't signed up yet)

4.5 Teacher Invitation Flow

6. Admin clicks 'Add Staff' → enters teacher email → clicks Send Invite
 7. Backend generates signed token → saves to Invitations collection → sends email via Mailtrap
 8. Teacher receives email with link: /signup?token=xxx&email=teacher@gmail.com&role=teacher
 9. Teacher fills name + password → account created → can now login
-

5. Teacher Panel

Role: teacher | Route prefix: /teacher | Access: Subject teachers

5.1 Teacher Sidebar Navigation

Nav Item	Route	Purpose
Home / Classes	/teacher/dashboard	Grid of assigned classes
All Exams	/teacher/exams	Cross-class exam overview table
Results	/teacher/results	View results per exam
Reports	/teacher/reports	AI-generated student reports
Add Student	/teacher/students	Student invitation management
Logout	—	End session

5.2 Teacher Home /teacher/dashboard

UI Element	Details
Class Cards Grid	Each card: Class Name, Subject, Student Count, Next Exam Date
Click on Card	Opens /teacher/class/:classId
No classes yet	Show empty state: 'No classes assigned yet. Contact your Admin.'

5.3 Class Detail /teacher/class/:classId

This page has 4 tabs. Class name shown at top with edit (pencil) icon.

Tab 1 — Overview

Element	Details
Class Name	Editable via pencil icon — PATCH /api/classes/:id
Description	Editable text area — save button
Class Code	Auto-generated 6-char code e.g. MTH4A2 — copy button
Class URL	Full shareable URL — copy button
Student Count	Live count of enrolled students

Tab 2 — Students

Section	Details
Pending Requests	Students who joined via code — show Name, Email, Requested Date, Accept / Reject buttons
Enrolled Students	Name, Email, Joined Date, Last Active, Remove button
Invite Student button	Teacher enters student email → sends invitation → student auto-enrolled on signup (no approval needed)

Tab 3 — Exams

Element	Details
Exam List	Title, Type (MCQ/Subjective/Mixed), Status badge, Scheduled Date, Actions
Status badges	Draft (gray), Scheduled (blue), Active (green), Completed (teal)
Create Exam button	Opens 4-step exam creation wizard
Actions	View Results Edit (only if Draft) Delete (only if Draft or Scheduled)

Exam Creation Wizard — 4 Steps

Step 1 — Exam Details (/teacher/class/:id/exam/create/step1):

- Exam Title — text input, required
- Subject / Course — text input, required
- Duration in minutes — number input, required
- Passing Marks — number input, required
- Next Step button → Step 2

Step 2 — Add Questions (/teacher/class/:id/exam/create/step2):

- Question Type dropdown: Multiple Choice (MCQ) / True-False / Short Answer (Subjective)
- Marks per question — number input
- Question Text — textarea
- For MCQ: 4 option fields + radio button to select correct answer
- For True-False: two radio options — True / False
- For Short Answer: just the question text (teacher marks manually)
- '+ Add Question' button — adds to list below
- Questions list shows below form — can delete individual questions
- 'Upload Question Bank' button — accepts CSV file with columns: type, question, optionA, optionB, optionC, optionD, correct, marks

Step 3 — Schedule Exam (/teacher/class/:id/exam/create/step3):

- Exam Date — date picker, required
- Start Time — time picker, required
- End Time — time picker, required (must be after start time)

- Randomize Questions toggle — shuffles question order per student

Step 4 — Review & Save (/teacher/class/:id/exam/create/step4):

- Full preview: title, subject, duration, passing marks, all questions listed
- Total marks auto-calculated and shown
- 'Save as Draft' button — saves with status: draft
- 'Publish Exam' button — saves with status: scheduled, students can see it

Tab 4 — Material

Element	Details
Upload button	Upload PDF, PPT, Word, image files — POST /api/materials
Materials grid	Each card: filename, upload date, file type icon, Download button, Delete button
Students see this	Same materials appear read-only on student's class page

5.4 All Exams Page — /teacher/exams

One table showing ALL exams across ALL classes this teacher has:

Column	Details
Exam Title	Clickable → /teacher/results/:examId
Class	Which class
Question Type	MCQ / Subjective / Mixed
Status	Color-coded badge
Scheduled Date & Time	Date + start time
Actions	View Results Edit Delete

5.5 Results Page — /teacher/results/:examId

Column	Details
Rank	1st, 2nd, 3rd... sorted by score descending

Column	Details
Student Name	Full name
Score	e.g. 22/25
Percentage	e.g. 88%
Grade	A+, A, B+, B, C, F — auto-calculated
Time Taken	How long student took
Cheating Flagged	Yes (red) / No (green)
Actions	View Detail button

- Failed students shown with red row highlight or separate 'Failed' section at bottom
- Export button: Download as CSV or PDF

5.6 Reports Page — /teacher/reports

Element	Details
Student selector	Dropdown — all students enrolled in teacher's classes
Generate Report button	Calls backend → AI analysis → returns report data
Subject-wise chart	Bar chart — avg score per subject
Summary cards	Avg Score, Pass Rate, Exams Taken, Cheating Incidents
AI Text Summary	Paragraph generated by AI: weak subjects, recommendations
Email to Parent button	Input field for parent email → sends report PDF to parent
Print / Download PDF	Download report as PDF



AI Implementation — Simple Way

In your Node.js backend, call the OpenAI or Claude API. Send: student name, subject scores, pass/fail per subject. Ask it to return a 2-3 paragraph summary identifying weak areas and recommendations. This counts as proper AI integration for your FYP.

6. Student Panel

Role: student | Route prefix: /student | Access: Enrolled students

6.1 How Student Account is Created

- Method 1 — Teacher invites via email: student gets link → /signup?token=x&email=x&role=student → fills name + password → auto-enrolled in that class
- Method 2 — Student joins via class code: student goes to /student/dashboard → clicks 'Join Class' → enters class code → request sent → teacher approves → enrolled

6.2 Student Sidebar Navigation

Nav Item	Route	Purpose
Home	/student/dashboard	Enrolled classes + join class + quick stats
My Results	/student/results	All exam results with analytics
Logout	—	End session

6.3 Student Home — /student/dashboard

UI Element	Details
Class Cards	Each enrolled class: Class Name, Teacher Name, Subject, Next Exam
Click on class card	Opens /student/class/:classId
Join Class button	Opens modal → enter class code → submit → pending request sent to teacher
Quick stats bar	Exams Completed Upcoming Exams Missing Exams

6.4 Student Class Detail — /student/class/:classId

This page has 2 tabs for students (not 4 like teacher):

Tab 1 — Exams

Element	Details
Upcoming Exams	Title, Date, Start Time, Duration, Status
Start Exam button	Only appears if current time is within exam's start-end window AND student hasn't attempted it yet

Element	Details
Completed Exams	Show score + grade for past exams
Missing Exams	Exams past end time with no attempt — shown with red 'Missed' badge

Tab 2 — Material

- Same materials teacher uploaded — student can view and download only
- No upload option for students

Leave Class

- 'Leave Class' button on class detail page
- Confirmation popup: 'Are you sure? Your exam history will be preserved.'
- DELETE /api/classes/:classId/leave

6.5 Live Exam Page — /student/exam/:examId

This is the most critical page. Cheating detection runs here.

Element	Details
Header bar	Exam title, Time remaining (countdown timer), Questions count
Question display	One question at a time OR all questions scrollable — your choice
MCQ options	Radio buttons — one selectable
True/False	Two radio buttons
Short Answer	Textarea input
Navigation	Previous / Next buttons, question number dots showing answered/unanswered
Submit button	Appears at last question — confirmation popup before submit
Auto-submit	When timer hits 0:00 OR 3 cheating violations — auto-submits

Cheating Detection — Must Implement

Add these event listeners when exam page mounts: (1) `document.addEventListener('visibilitychange')` — fires when tab is switched. (2) `window.addEventListener('blur')` — fires when window loses focus. Each violation: show warning popup + POST `/api/exam-attempts/:id/flag` to backend. On 3rd violation: auto-submit exam + mark as cheating detected.

```
// ExamPage.jsx — Cheating Detection
const [violations, setViolations] = useState(0);

useEffect(() => {
```

```
const flag = async () => {
  const count = violations + 1;
  setViolations(count);
  await axios.post(`/api/exam-attempts/${attemptId}/flag`);
  if (count >= 3) {
    await submitExam(); // auto submit
  } else {
    showWarning(`Warning ${count}/3: Tab switch detected!`);
  }
};
document.addEventListener('visibilitychange', () => {
  if (document.hidden) flag();
});
window.addEventListener('blur', flag);
return () => { /* cleanup listeners */ };
}, [violations]);
```

6.6 My Results Page — /student/results

UI Element	Details
Stats cards row	Average Score (%), Exams Completed, Missing Exams, Avg Time per Exam
Filter dropdown	'Filter by: All Classes' → can filter by specific class
Recent Exams table	Exam Name, Date, Score (e.g. 22/25), Grade badge, View Analysis button
View Analysis link	Opens /student/results/:examId/analysis

6.7 Exam Analysis Page — /student/results/:examId/analysis

Section	Details
Score summary	Score, Percentage, Grade, Time Taken, Pass/Fail badge
Question breakdown	Each question: your answer, correct answer, Correct (green tick) or Wrong (red cross)
Performance chart	Pie chart or bar chart: correct vs incorrect vs skipped
Class comparison	Your score vs class average — simple bar

7. Complete URL Routes Reference

All routes in the application — for frontend React Router and backend Express.

7.1 Public Routes (No Auth Required)

Route	Page	Component
/login	Login page	LoginPage
/signup	Invitation signup	SignupPage
*	404 Not Found	NotFoundPage

7.2 SuperAdmin Routes (role: superadmin)

Route	Page
/superadmin/dashboard	SuperAdmin home — platform stats
/superadmin/admins	All admins list
/superadmin/admins/create	Create new admin form
/superadmin/admins/:adminId	Admin detail view
/superadmin/settings	SuperAdmin profile settings

7.3 Admin Routes (role: admin)

Route	Page
/admin/dashboard	Admin home — institution stats
/admin/teachers	Teachers list + invite
/admin/students	All students view
/admin/settings	Institution + account settings

7.4 Teacher Routes (role: teacher)

Route	Page
/teacher/dashboard	Teacher home — class grid
/teacher/class/:classId	Class detail — 4 tabs
/teacher/class/:classId/exam/create	Exam creation wizard (steps 1-4)
/teacher/exams	All exams overview

Route	Page
/teacher/results/:examId	Exam results for a class
/teacher/reports	AI student reports
/teacher/students	Student invitation management

7.5 Student Routes (role: student)

Route	Page
/student/dashboard	Student home — class cards
/student/class/:classId	Class detail — 2 tabs
/student/exam/:examId	Live exam page with anti-cheat
/student/results	My results — all exams
/student/results/:examId/analysis	Per-exam analysis detail

8. MongoDB Collections & Fields

Collection	Key Fields
users	_id, name, email, passwordHash, role (superadmin/admin/teacher/student), institutionId (null for superadmin), isActive, createdAt
institutions	_id, name, adminId, createdAt — (one institution per admin)
invitations	_id, email, role, token (hashed), expiresAt, used (bool), sentBy (userId), classId (for student invites)
classes	_id, name, description, subject, classCode, teacherId, institutionId, studentIds[], createdAt
joinRequests	_id, classId, studentId, status (pending/approved/rejected), requestedAt
exams	_id, classId, teacherId, title, subject, duration, passingMarks, questions[], scheduleDate, startTime, endTime, randomize, status (draft/scheduled/active/completed), totalMarks
examAttempts	_id, examId, studentId, answers[], score, percentage, grade, timeTaken, cheatingFlags (array of timestamps), violationCount, status (in-progress/submitted/auto-submitted), submittedAt
materials	_id, classId, teacherId, filename, fileUrl, fileType, size, uploadedAt

 **institutionId Field**

Every teacher and student document should have an institutionId field pointing to their institution. This ensures Admin can only query their own institution's data. SuperAdmin queries without this filter to see everything.

9. Key Backend API Endpoints

9.1 Auth

Method	Endpoint	Access	Purpose
POST	/api/auth/login	Public	Login — returns JWT token
POST	/api/auth/signup-invite	Token only	Create account from invitation
POST	/api/auth/logout	Authenticated	Clear session

9.2 SuperAdmin

Method	Endpoint	Purpose
GET	/api/superadmin/stats	Platform-wide counts
GET	/api/superadmin/admins	All admins list
POST	/api/superadmin/admins	Create new admin
PATCH	/api/superadmin/admins/:id/suspend	Suspend admin
DELETE	/api/superadmin/admins/:id	Delete admin

9.3 Exams & Attempts

Method	Endpoint	Purpose
POST	/api/exams	Create exam (teacher)
GET	/api/exams/:examId	Get exam details

Method	Endpoint	Purpose
POST	/api/exam-attempts	Student starts exam — creates attempt record
PATCH	/api/exam-attempts/:id/submit	Student submits answers
POST	/api/exam-attempts/:id/flag	Log cheating violation
GET	/api/exams/:examId/results	Teacher views all results for an exam